

*****Pornind de la aplicația exemplu cu Okazii (simulat evident) din laboratorul 7, să se modifice aceasta astfel încât să se introducă un procesor de flux care să realizeze statistici cu privire la numărul de mesaje care au fost necesare pentru o licitație până la adjudecare și apoi să scrie aceste informații într-un fișier local.

Dacă se consideră necesar se mai pot crea oricâte servicii se dorește conform gândirii studentului. Se vor respecta principiile SOLID pentru microservicii. La prezentare se va arăta diagrama de clase și se va explica aceasta împreună cu gradul de respectare a principiilor. Apoi va urma discuția pe cod. +1

FA SET UP UL AI INDICATII IN LAB 7 LA FEL PT COMPILARE/EXECUTIE
MAI INTAI SA MEARGA EXEMPLU SI DUPA RESTU

Eu presupun ca in lab 7 okazii AuctioneerMicroservice
la functia receiveBidsSubscription putem numara cate mesaje din de la bidder
punem intr un fisier

astea in clasa

```
private var messageCount = 0  
private val receivedMessages = mutableListOf<String>()
```

Poti adauga functia asta in fisierul AuctioneerMicroservice

```
private fun writeMessagesToFile() {  
    val outputFile = File("output.txt")  
  
    try {  
        val writer = FileWriter(outputFile, true) // Deschideți fișierul pentru a adăuga conținut la el  
  
        writer.append("S-a incheiat licitatie. Numar total de mesaje: $messageCount\n")  
  
        for (message in receivedMessages) {  
            writer.append(message).append("\n")  
        }  
  
        writer.close()  
    } catch (e: IOException) {  
        e.printStackTrace()  
    }  
}
```

linia asta trebuie adaugata la functia receiveBids la onComplete
writeMessagesToFile()

```
liniile asta trebuie adaugate la receiveBids onNext  
messageCount++  
receivedMessages.add(message.toString())
```

```
-----  
pt mai multi de o data  
for i in {1..5}; do java -jar BidderMicroservice_jar/BidderMicroservice.jar & done
```

```
-----  
GASIT, modul nou in care faci necessary.kt
```

```
import io.reactivex.rxjava3.core.Observable  
import io.reactivex.rxjava3.kotlin.subscribeBy  
import java.io.BufferedReader  
import java.io.FileOutputStream  
import java.io.InputStreamReader  
import java.io.PrintWriter  
import java.net.ServerSocket  
import java.net.Socket  
import java.net.SocketTimeoutException  
import java.time.LocalDateTime
```

```
class NecessaryMessageProcessor{  
    private var necessaryMessageSocket: ServerSocket  
    private var receiveBidsObservable: Observable<String>  
    private val bidderConnections: MutableList<Socket> = mutableListOf()  
    private val messageList: MutableList<String> = mutableListOf()  
  
    companion object Constants {  
        const val NECESSARYMESSAGE_PORT = 2121  
        const val DURATION: Long = 20_000 // licitatie dureaza 15 secunde  
    }  
    init{  
        necessaryMessageSocket = ServerSocket(NECESSARYMESSAGE_PORT)  
        necessaryMessageSocket.setSoTimeout(DURATION.toInt())  
        println("NecessaryMessageMicroservice se executa pe portul:  
${necessaryMessageSocket.localPort}")  
        println("Se asteapta mesaje de eroare...")  
  
        receiveBidsObservable = Observable.create<String> { emitter ->  
            while (true) {  
                try {  
                    val bidderConnection = necessaryMessageSocket.accept()
```

```

        bidderConnections.add(bidderConnection)

        // se citeste mesajul de la bidder de pe socketul TCP
        val bufferedReader =
BufferedReader(InputStreamReader(bidderConnection.inputStream))
        val receivedMessage = bufferedReader.readLine()

        // daca se primeste un mesaj gol (NULL), atunci inseamna ca cealalta parte a
socket-ului a fost inchisa
        if (receivedMessage == null) {
            // deci subscriber-ul respectiv a fost deconectat
            bufferedReader.close()
            bidderConnection.close()
        }

        // se emite ce s-a citit ca si element in fluxul de mesaje
        emitter.onNext(receivedMessage)
    } catch (e: SocketTimeoutException) {
        // daca au trecut cele 15 secunde de la pornirea licitatiei, inseamna ca licitatia s-a
incheiat
        // se emite semnalul Complete pentru a incheia fluxul de oferte
        emitter.onComplete()
        break
    }
}
}
}
}

private fun receiveBids() {
    // se incepe prin a primi ofertele de la bidderi
    receiveBidsObservable.subscribeBy(
        onNext = {
            val message = Message.deserialize(it.toByteArray())
            messageList.add(message.toString())
            println(message)
        },
        onComplete = { },
        onError = {
            writeError("Eroare: $it")
            println("Eroare: $it")
        }
    )
}
}

```

```

fun write(line: String) {
    FileOutputStream("../././file.txt", true).bufferedWriter().use { writer ->
        writer.append("${LocalDateTime.now()} --> $line\n")
    }
}

fun writeError(error: String) {
    write(error)
}

fun run() {
    receiveBids()
}

}

fun main(args: Array<String>) {
    val necessaryMessageProcessor = NecessaryMessageProcessor()
    necessaryMessageProcessor.run()
}

---- au aparut modificari si la BidderMicroservice
import io.reactivex.rxjava3.core.Observable
import io.reactivex.rxjava3.kotlin.subscribeBy
import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.Socket
import kotlin.Exception
import kotlin.random.Random
import kotlin.system.exitProcess

class BidderMicroservice {
    private var auctioneerSocket: Socket
    private var necessaryMessageSocket: Socket
    private var auctionResultObservable: Observable<String>
    private var myIdentity: String = "[BIDDER_NECONNECTAT]"

    companion object Constants {
        const val AUCTIONEER_HOST = "localhost"
        const val AUCTIONEER_PORT = 1500
        const val NECESSARYMESSAGE_PORT = 2121
        const val MAX_BID = 10_000
        const val MIN_BID = 1_000
    }

    init {

```

```

try {
    auctioneerSocket = Socket(AUCTIONEER_HOST, AUCTIONEER_PORT)
    println("M-am conectat la Auctioneer!")
    necessaryMessageSocket = Socket(AUCTIONEER_HOST,
NECESSARYMESSAGE_PORT)
    println("M-am conectat la NecessaryMessage!")

    myIdentity = "[${auctioneerSocket.localPort}]"

    // se creeaza un obiect Observable ce va emite mesaje primite printr-un TCP
    // fiecare mesaj primit reprezinta un element al fluxului de date reactiv
    auctionResultObservable = Observable.create<String> { emitter ->
        // se citeste raspunsul de pe socketul TCP
        val bufferedReader =
BufferedReader(InputStreamReader(auctioneerSocket.inputStream))
        val receivedMessage = bufferedReader.readLine()

        // daca se primeste un mesaj gol (NULL), atunci inseamna ca cealalta parte a
socket-ului a fost inchisa
        if (receivedMessage == null) {
            bufferedReader.close()
            auctioneerSocket.close()

            emitter.onError(Exception("AuctioneerMicroservice s-a deconectat."))
            return@create
        }

        // mesajul primit este emis in flux
        emitter.onNext(receivedMessage)

        // deoarece se asteapta un singur mesaj, in continuare se emite semnalul de
incheiere al fluxului
        emitter.onComplete()

        bufferedReader.close()
        auctioneerSocket.close()
    }
} catch (e: Exception) {
    sendError(java.lang.Exception("$myIdentity Nu ma pot conecta la Auctioneer!"))
    println("$myIdentity Nu ma pot conecta la Auctioneer!")
    exitProcess(1)
}
}

```

```

private fun bid() {
    // se genereaza o oferta aleatorie din partea bidderului curent
    val pret = Random.nextInt(MIN_BID, MAX_BID)

    // se creeaza mesajul care incapsuleaza oferta
    val biddingMessage =
        Message.create("${auctioneerSocket.localAddress}:${auctioneerSocket.localPort}",
            "licitez $pret")

    // bidder-ul trimite pretul pentru care doreste sa liciteze
    val serializedMessage = biddingMessage.serialize()
    auctioneerSocket.getOutputStream().write(serializedMessage)
    necessaryMessageSocket.getOutputStream().write(serializedMessage)

    // exista o sansa din 2 ca bidder-ul sa-si trimita oferta de 2 ori, eronat
    if (Random.nextBoolean()) {
        auctioneerSocket.getOutputStream().write(serializedMessage)
    }
}

private fun waitForResult() {
    println("$myIdentity Astept rezultatul licitatiei...")
    // bidder-ul se inscrie pentru primirea unui raspuns la oferta trimisa de acesta
    val auctionResultSubscription = auctionResultObservable.subscribeBy(
        // cand se primeste un mesaj in flux, inseamna ca a sosit rezultatul licitatiei
        onNext = {
            val resultMessage: Message = Message.deserialize(it.toByteArray())
            println("$myIdentity Rezultat licitatie: ${resultMessage.body}")
        },
        onError = {
            sendError(it as Exception)
            println("$myIdentity Eroare: $it")
        }
    )
}

// se elibereaza memoria obiectului Subscription
auctionResultSubscription.dispose()
}

private fun sendError(error: Exception) {
    necessaryMessageSocket = Socket("localhost", 2121)
    val err = Message.create(
        "${auctioneerSocket.localAddress}:${auctioneerSocket.localPort}",
        "${auctioneerSocket.localAddress}:${auctioneerSocket.localPort}:${error.message}"
    )
}

```

```

        )
        necessaryMessageSocket.getOutputStream().write(err.serialize())
    }

    fun run() {
        bid()
        waitForResult()
    }
}

fun main(args: Array<String>) {
    val bidderMicroservice = BidderMicroservice()
    bidderMicroservice.run()
}

----- modificari si la AuctioneerMicroservice
import io.reactivex.rxjava3.core.Observable
import io.reactivex.rxjava3.disposables.CompositeDisposable
import io.reactivex.rxjava3.kotlin.subscribeBy
import io.reactivex.rxjava3.plugins.RxJavaPlugins
import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.ServerSocket
import java.net.Socket
import java.net.SocketTimeoutException
import java.util.*
import kotlin.collections.ArrayList
import kotlin.system.exitProcess

class AuctioneerMicroservice {
    private lateinit var auctioneerSocket: ServerSocket
    private lateinit var messageProcessorSocket: Socket
    private lateinit var necessaryMessageSocket: Socket
    private var receiveBidsObservable: Observable<String>
    private val subscriptions = CompositeDisposable()
    private val bidQueue: Queue<Message> = LinkedList<Message>()
    private val bidderConnections: MutableList<Socket> = mutableListOf()

    companion object Constants {
        const val MESSAGE_PROCESSOR_HOST = "localhost"
        const val MESSAGE_PROCESSOR_PORT = 1600
        const val AUCTIONEER_PORT = 1500
        const val AUCTION_DURATION: Long = 15_000 // licitatie dureaza 15 secunde
    }
}

```

```

init {

    // se creeaza obiectul Observable cu care se genereaza evenimente cand se primesc
    oferte de la bidder
    receiveBidsObservable = Observable.create<String> { emitter ->
        try {
            auctioneerSocket = ServerSocket(AUCTIONEER_PORT)
            auctioneerSocket.setSoTimeout(AUCTION_DURATION.toInt())
            println("AuctioneerMicroservice se executa pe portul: ${auctioneerSocket.localPort}")
            println("Se asteapta oferte de la bidder...")
        }
        catch (e: Exception) {
            emitter.onError(e)
        }
        // se asteapta conexiuni din partea bidderilor
        while (true) {
            try {
                val bidderConnection = auctioneerSocket.accept()
                bidderConnections.add(bidderConnection)

                // se citeste mesajul de la bidder de pe socketul TCP
                val bufferReader =
                BufferedReader(InputStreamReader(bidderConnection.inputStream))
                val receivedMessage = bufferReader.readLine()

                // daca se primeste un mesaj gol (NULL), atunci inseamna ca cealalta parte a
                socket-ului a fost inchisa
                if (receivedMessage == null) {
                    // deci subscriber-ul respectiv a fost deconectat
                    bufferReader.close()
                    bidderConnection.close()

                    emitter.onError(Exception("Eroare: Bidder-ul ${bidderConnection.port} a fost
                    deconectat."))
                }

                // se emite ce s-a citit ca si element in fluxul de mesaje
                emitter.onNext(receivedMessage)
            } catch (e: SocketTimeoutException ) {
                // daca au trecut cele 15 secunde de la pornirea licitatiei, inseamna ca licitatia s-a
                incheiat

                // se emite semnalul Complete pentru a incheia fluxul de oferte
                emitter.onComplete()
                break
            }
        }
    }
}

```



```

        } catch (e: Exception) {
            emitter.onError(e)
            break
        }
    }
}

```

```

private fun receiveBids() {
    // se incepe prin a primi ofertele de la bidderi
    val receiveBidsSubscription = receiveBidsObservable.subscribeBy(
        onNext = {
            val message = Message.deserialize(it.toByteArray())
            println(message)
            bidQueue.add(message)
        },
        onComplete = {
            // licitatia s-a incheiat
            // se trimit raspunsurile mai departe catre procesorul de mesaje
            println("Licitatia s-a incheiat! Se trimit ofertele spre procesare...")
            forwardBids()
        },
        onError = {
            sendError(it as Exception)
            println("Eroare: ${it.message}")
        }
    )
    subscriptions.add(receiveBidsSubscription)
}

```

```

private fun forwardBids() {
    try {
        messageProcessorSocket = Socket(MESSAGE_PROCESSOR_HOST,
MESSAGE_PROCESSOR_PORT)
        subscriptions.add(Observable.fromIterable(bidQueue).subscribeBy(
            onNext = {
                // trimitere mesaje catre procesorul de mesaje
                messageProcessorSocket.getOutputStream().write(it.serialize())
                println("Am trimis mesajul: $it")
            },
            onComplete = {
                println("Am trimis toate ofertele catre MessageProcessor.")
                val bidEndMessage = Message.create(

```

```

"${messageProcessorSocket.localAddress}:${messageProcessorSocket.localPort}",
    "final"
)
messageProcessorSocket.getOutputStream().write(bidEndMessage.serialize())

// dupa ce s-a terminat licitatia, se asteapta raspuns de la
MessageProcessorMicroservice
// cum ca a primit toate mesajele
val bufferedReader =
BufferedReader(InputStreamReader(messageProcessorSocket.inputStream))
bufferedReader.readLine()

messageProcessorSocket.close()

finishAuction()
}
))
} catch (e: Exception) {
println("Nu ma pot conecta la MessageProcessor!")

auctioneerSocket.close()
exitProcess(1)
}
}

private fun sendError(error: Exception) {
    necessaryMessageSocket = Socket("localhost", 2121)
    val err = Message.create(
        "${messageProcessorSocket.localAddress}:${messageProcessorSocket.localPort}",
        "${messageProcessorSocket.localAddress}:${messageProcessorSocket.localPort}:${error.message}"
    )
    necessaryMessageSocket.getOutputStream().write(err.serialize())
}

private fun finishAuction() {
    // se asteapta rezultatul licitatiei
    try {
        val biddingProcessorConnection = auctioneerSocket.accept()
        val bufferedReader =
BufferedReader(InputStreamReader(biddingProcessorConnection.inputStream))

```

```

// se citeste rezultatul licitatiei de la AuctioneerMicroservice de pe socketul TCP
val receivedMessage = bufferedReader.readLine()

val result: Message = Message.deserialize(receivedMessage.toByteArray())
val winningPrice = result.body.split(" ")[1].toInt()
println("Am primit rezultatul licitatiei de la BiddingProcessor: ${result.sender} a castigat
cu pretul: $winningPrice")

// se creeaza mesajele pentru rezultatele licitatiei
val winningMessage = Message.create(auctioneerSocket.localSocketAddress.toString(),
    "Licitatie castigata! Pret castigator: $winningPrice")
val losingMessage = Message.create(auctioneerSocket.localSocketAddress.toString(),
    "Licitatie pierduta...")

// se anunta castigatorul
bidderConnections.forEach {
    if (it.remoteSocketAddress.toString() == result.sender) {
        it.getOutputStream().write(winningMessage.serialize())
    } else {
        it.getOutputStream().write(losingMessage.serialize())
    }
    it.close()
}
} catch (e: Exception) {
    println("Nu ma pot conecta la BiddingProcessor!")
    auctioneerSocket.close()
    exitProcess(1)
}

// se elibereaza memoria din multimea de Subscriptions
subscriptions.dispose()
}

fun run() {
    receiveBids()
}

fun main(args: Array<String>) {
    val bidderMicroservice = AuctioneerMicroservice()
    bidderMicroservice.run()
}

```